Department of Computer Science, University College London

COMP0222/COMP0249 Lab 9: RGBD Camera

14th March, 2026 ver. 20260314

1 Scope and Purpose

In this lab, you will explore 3D spatial data using the Intel RealSense D455. The D455 uses stereoscopic vision to provide a dense 3D point cloud. You will implement a data logging framework to capture synchronized RGB and Depth frames, ensuring they are spatially aligned.

You will distinguish between raw sensor data and processed metadata, utilizing the camera's global shutter to capture motion-blur-free imagery. Finally, you will explore the fundamentals of 3D reconstruction by preparing your captured data for a Structure-from-Motion (SfM) pipeline using COLMAP. This enables building full 3D digital models.

2 Scenario

Motion and mapping in 3D space are characterized by 6 Degrees of Freedom (DoF), consisting of translation (x, y, z) and rotation (roll, pitch, yaw). In this lab, you will perform data logging to build a dataset for 3D reconstruction. 3D sensing allows for the detection of complex geometries, such as table surfaces, hanging obstacles, and terrain irregularities. By aligning the depth map with the RGB sensor, you create a "RGB-D" image where every color pixel is associated with a real-world distance. This technique is fundamental for spatial computing, robotic manipulation, and large-scale architectural scanning.

3 Sensor Specification: Intel RealSense D455

The Intel RealSense D455 is a stereoscopic depth camera designed for long-range and high-precision applications, see Fig. 1. It operates based on the principle of Stereo Vision with Infrared Projector assistance.

- **Mechanism: Two global shutter sensors (separated by a 95mm baseline) calculate depth via disparity. An infrared projector helps the sensors "see" texture on flat, featureless surfaces (like white walls).**
- **Synchronization: An internal hardware clock ensures that the RGB and Depth frames are captured at the exact same moment.**
- **Key Specifications:**
  - **Ideal Range: 0.6m – 6m (Depth can reach up to 20m)**
  - **Sensor Type: Global Shutter (minimizes motion blur)**
  - **Field of View (H $\times$ V):** $86^\circ \times 57^\circ$
  - **RGB Resolution: Up to** 1280×800 **at 30fps**
  - **Connection: USB 3.1 Gen 1 (Type-C)**

Figure 1: Intel RealSense D455 Depth Camera.

4 Requirements

This lab requires Python 3.8+. You must create a virtual environment to manage hardware-specific libraries. The RealSense SDK (`pyrealsense2`) is highly sensitive to binary compatibility; using a `venv` prevents conflicts with system-level packages (see the next section for venv).
Note: Ensure your computer is equipped with a USB 3.0/3.1 port. A USB 2.0 port will severely limit resolution and framerate.

5 Activities

Activity 0: Basic Connectivity and Environment Setup

- **Objective: Establish communication with the Intel RealSense D455 and isolate the development environment to ensure library compatibility.**
- **Instructions:**
  - **Step 1: Driver and SDK Installation**
    - **Windows: Download the latest** `Intel.RealSense.SDK-Win10.exe` **from the Official GitHub Releases: https://github.com/IntelRealSense/librealsense/releases/latest. Run the installer to set up the RealSense Viewer and USB drivers.**
    - **Ubuntu: Register the Intel repository and install via** `apt` **using the following commands:**

```
sudo apt install librealsense2-dkms librealsense2-utils
sudo apt install librealsense2-dev librealsense2-dbg
```

  - **Step 2: Firmware Verification Launch the** `RealSense Viewer`**. If a "Firmware Update Available" notification appears in the top-right corner, proceed with the update to ensure the D455's global shutter and IMU are properly calibrated.**
  - **Step 3: Virtual Environment Setup Open your project folder and initialize the Python environment:**

```
python -m venv rs_env
source rs_env/bin/activate  # Linux/Mac
rs_env\Scripts\activate     # Windows
pip install pyrealsense2 opencv-python numpy
```

Once the viewer is installed, run it (see Fig. 2). Explore the 2D and 3D options, see the top right corner.

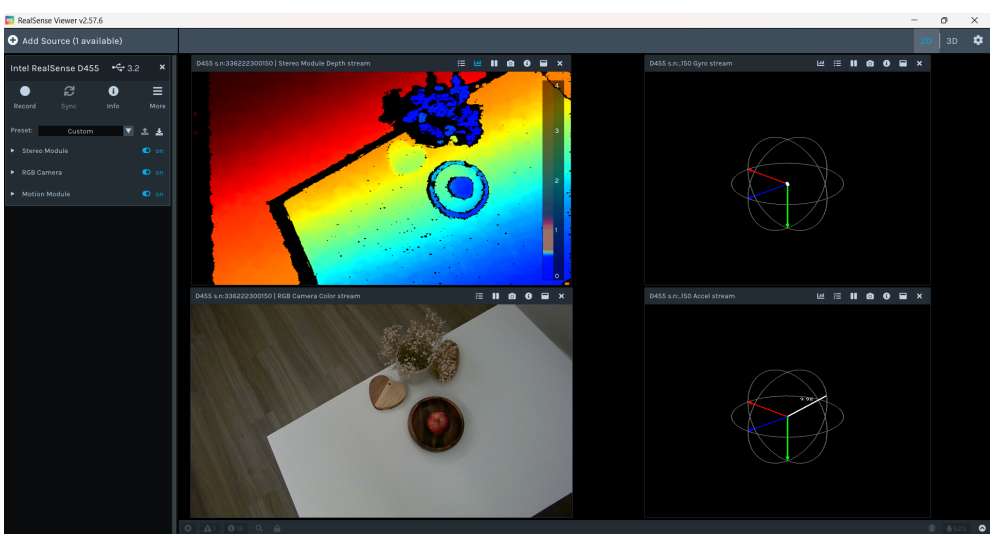

Figure 2: Intel D455 viewer.

Activity 1: Real-Time Aligned Data Logging

- **Objective: Capture synchronized RGB-D pairs where the depth map is aligned to the RGB camera's coordinate system.**
- **Instructions:**
  - **Run the** `d455_logger.py` **script. Use this file for controlled image capturing. This is suitable for SfM.**
  - **Run the** `d455_logger_stream.py` **script. Use this file to capture images continuously. This is suitable for SLAM.**
  - **This script uses the** `rs.align` **object to mathematically project the depth data onto the RGB lens's viewpoint.**
  - **Press SPACE to save. Press ESC to exit the program.**
- **Data Output:**
  - `.png`**: Lossless RGB image.**
  - `.npy`**: Raw 16-bit NumPy array containing depth in millimeters.**
  - `.csv`**: Metadata log containing hardware exposure and gain values.**

Activity 2: Data Playback and Quality Inspection

- **Objective: Review the logged 3D data and verify that the RGB-Depth alignment is correct.**
- **Instructions:**
  - **Run the** `d455_browser.py` **script. Make sure to specify the folder name inside this file, at the top of the file.**
  - **Follow the instructions printed on the terminal to replay your recording.**
  - **Observe the depth shadows, i.e. areas where the camera could not see (represented as black pixels). Ensure your target objects are not within these shadow regions.**

Activity 3: Dataset Preparation for 3D Reconstruction (COLMAP)

- **Objective: Export captured data for processing in an external Structure-from-Motion (SfM) pipeline.**
- **Instructions:**
  - **Isolate the** `.png` **files into an** `/images` **directory.**
  - **Load the directory into COLMAP.**
  - **Perform Feature Extraction and Sequential Matching.**
  - **Because the D455 uses a global shutter, features will remain sharp even if the camera was moving during capture, resulting in a higher-quality 3D point cloud.**
  - **Try two sets of data: a small scale from a desktop object, a large scale environment, e.g. a room-sized environment.**

Activity 4: Real-Time Volumetric Mapping with TSDF

- **Objective: Implement a real-time mapping and localization pipeline that integrates RGB-D data into a scalable truncated signed distance field (TSDF) volume using frame-to-model, aka frame-to-map, tracking.**
- **Instructions:**
  - **Need Open3D for this lab:** `pip install open3d`
  - **For more information, please see https://www.open3d.org/docs/release/getting_started.html**
  - **Explore the code provided:** `d455_TSDF.py`
  - **Voxel Grid Initialization: Observe that in the code provided, we set the** `voxel_length` **to** `0.03` **(3cm). Discuss how decreasing this value impacts CPU overhead versus reconstruction detail.**
  - **Frame-to-Model ICP: Unlike frame-to-frame odometry, here we align the current** `PointCloud` **to the global** `TSDF` **map using Point-to-Plane ICP. This is similar to what you did with the Lidar mapping.**
  - **Tracking Robustness: Observe the Fitness Score. If the camera moves too quickly or faces a featureless wall, the tracking will slip. Move the D455 slowly to maintain a fitness score above** `0.5`**.**
  - **Visualization: The provided code implements a live visualizer that displays:**
    - **The accumulating 3D mesh.**
    - **The real-time camera trajectory (Green LineSet).**
    - **The current camera frustum (Red pyramid).**
  - **While the camera is connected to your device, run the code provided:** `d455_TSDF.py`
  - **Analysis: Compare the drift in a loop (returning to the start point). Does the double-wall effect reappear if the environment is poorly lit?**

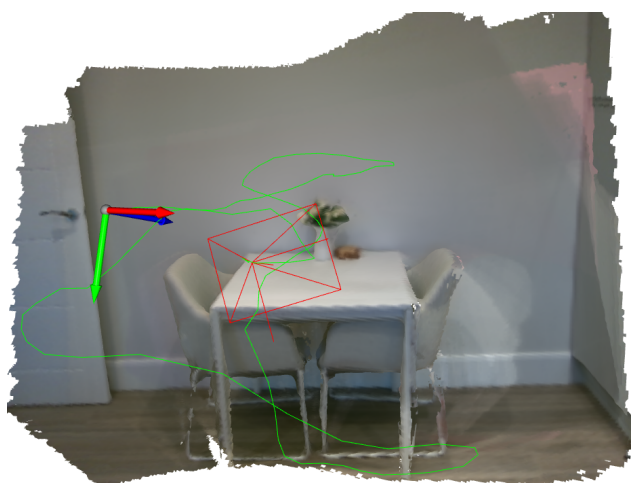

Figure 3: A TSDF map: the green trajectory shows the path of the camera, the red frame shows the current camera pose, the 3D frame shows the start position.

Discussion Questions

1. **Voxel Resolution vs. Performance: During the activity, you set the `voxel_length` to 0.03. If you were to change this to 0.005 (5mm) to capture fine details on a desk, what would you expect to happen to the live feed frame rate? Why is TSDF integration computationally expensive on a CPU as the map grows?**

2. **The Geometric Feature Requirement: Try pointing the camera at a large, perfectly flat, and featureless white wall while moving sideways. Does the tracking stay locked, or does the camera frustum slide in the visualizer? Explain why ICP struggles with planar symmetry.**

3. **Impact of `sdf_trunc`: The `sdf_trunc` parameter defines how far from a surface the algorithm cares about the distance values. If this value is set too high (e.g., 0.5m), how might that affect the sharpness of corners and the thickness of reconstructed walls?**

4. **Global Shutter Advantage: The Intel D455 uses a global shutter for its RGB and depth sensors, whereas many smartphones use rolling shutters. Why is a global shutter critical for the frame-to-model registration you implemented, especially when the camera is rotating?**

5. **Drift and Loop Closure: If you map a long hallway and then return to your exact starting point, you may notice the start and end of the hallway do not perfectly align. This is known as drift. Aside from moving slower, what additional mathematical step could be added to the pipeline to snap these two views together?**

A Technical Appendix: Volumetric Localization, Mapping, and TSDF

A.1 Understanding TSDF (Truncated Signed Distance Function)

The TSDF is the industry-standard method for dense volumetric 3D reconstruction. Unlike a point cloud, which only stores discrete (x, y, z) coordinates, a TSDF represents the environment as a voxel grid: a 3D array of cubes (voxels).

Each voxel in the grid stores two primary values:

1. $F(\mathbf{x})$ (The Signed Distance): The distance from the center of the voxel to the nearest estimated surface (See Fig. 4).
   - Positive: The voxel is in front of the surface (in empty space).
   - Negative: The voxel is behind the surface (inside an object).
   - Zero: The voxel is exactly on the surface.
2. $W(\mathbf{x})$ (The Weight): A confidence value that increases as more camera frames observe and validate that specific voxel.

A.1.1 The Concept of Truncation

In a raw signed distance function (SDF), every voxel in the scene would store a distance value. To optimize computation and memory, we use Truncation (μ). If a voxel is further than μ from a surface, its value is capped. This ensures the algorithm only processes data in a narrow band around actual surfaces, effectively filtering out background noise and sensor outliers (See Fig. 4).

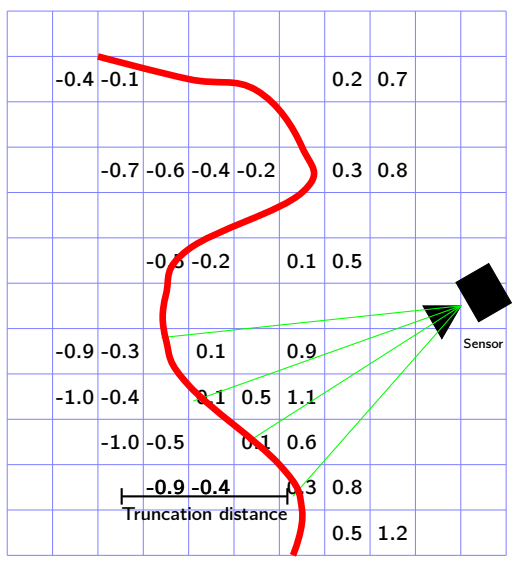

Figure 4: 2D visualization of the truncated signed distance function (TSDF) mapping. The diagram illustrates how distance values are stored in a voxel grid: positive values indicate empty space, while negative values indicate the interior of an object. The zero-crossing (thick red line) defines the reconstructed surface relative to the sensor position.

A.1.2 Volumetric Integration Math

As new RGB-D frames arrive, the global grid is updated using a running weighted average. For a voxel at position x, the update rule for the distance D and weight W is:

$$D_{i+1}(\mathbf{x}) = \frac{W_i(\mathbf{x})D_i(\mathbf{x}) + w_{i+1}(\mathbf{x})d_{i+1}(\mathbf{x})}{W_i(\mathbf{x}) + w_{i+1}(\mathbf{x})} \tag{1}$$

Where d_{i+1} is the depth measurement from the current frame. This recursive filtering is what allows the TSDF to produce smooth, noise-free surfaces from the raw, flickering depth data of the Intel D455.

A.2 The Open3D Library

Open3D is an open-source library specifically designed for rapid development of 3D data processing software. It is highly optimized for both CPU and GPU (via the Tensor API) operations.

- Scalable TSDF Volumes: Open3D uses a hierarchical hash map to manage voxels. Instead of pre-allocating a massive 3D array (which would exhaust RAM), it only allocates memory for voxels where a surface is actually detected. This allows for room-scale mapping at high resolutions.
- Hybrid Odometry: The `compute_rgbd_odometry` function minimizes both photometric error (color consistency) and geometric error (depth consistency) simultaneously to estimate camera motion.
- Point-to-Plane ICP: By utilizing Surface Normals (the vector perpendicular to a surface), Open3D's ICP implementation allows for "sliding" alignment. This is significantly more robust for indoor environments (walls and floors) than standard point-to-point methods.

A.3 Surface Extraction (Marching Cubes)

To visualize the TSDF volume as a 3D model, the Marching Cubes algorithm is applied. The algorithm inspects the signed distance values at the corners of each voxel. If a voxel has both positive and negative distances at its vertices, a surface must pass through it (the zero-crossing). The algorithm then "marches" through the grid, creating triangular faces to represent the surface.

A.4 Parameter Reference Table

The following table summarizes the key parameters used in the Python implementation for the Intel D455 camera.

Table 1: Localization, Mapping, and TSDF Parameters

Parameter	Recommended Value	Description
`voxel_length`	0.02 – 0.05m	The size of each 3D cube in the grid.
`sdf_trunc`	0.04 – 0.08m	The distance around a surface to integrate.
`depth_trunc`	3.0m	Ignores depth data beyond this distance.
`depth_scale`	1000.0	Converts RealSense 16-bit integers to meters.
`fitness_score`	> 0.5	Threshold for a good alignment match.

A.5 References

- Lorensen, William E. and Cline, Harvey E. "Marching cubes: A high resolution 3D surface construction algorithm," ACM SIGGRAPH Computer Graphics, Volume 21, Issue 4, pp. 163 - 169, 1987, https://dl.acm.org/doi/10.1145/37402.37422

- Newcombe, Richard A. and Izadi, Shahram and Hilliges, Otmar and Molyneaux, David and Kim, David and Davison, Andrew J. and Kohi, Pushmeet and Shotton, Jamie and Hodges, Steve and Fitzgibbon, Andrew, "KinectFusion: Real-time dense surface mapping and tracking," IEEE International Symposium on Mixed and Augmented Reality (ISMAR) pp. 127-136, 2011, https://ieeexplore.ieee.org/document/6162880